

OOP কেন লাগে?

1. শুধু **Variable** ব্যবহার করলে **data** ছড়িয়ে যায়

```
let studentName = "Rafi";  
let studentRoll = 27;  
let studentMarks = 78;
```

একজন student-এর information আলাদা আলাদা variable-এ আছে।

১০ জন student হলে অনেক variable হয়ে যাবে।

Problem: কোন variable কোন student-এর, সেটা language-এর কাছে clear না।

2. **Object related data** একসাথে রাখে

```
const rafi = {  
  name: "Rafi",  
  roll: 27,  
  marks: 78  
};
```

এখন:

```
rafi  
├── name  
├── roll  
└── marks
```

অর্থাৎ:

Object = related data একসাথে রাখার একটা **box/folder**।

`rafi.name` মানে Rafi object-এর `name` property।

Object কী কী **problem solve** করে?

✅ **Problem 1: Grouping**

আগে:

```
studentName  
studentRoll
```

studentMarks

এখন:

```
student
├── name
├── roll
└── marks
```

সব related data একসাথে।

কিন্তু **Object**-এরও ৩টা সমস্যা আছে

❌ 1. Structure guaranteed না

```
{
  diver: "Kamal" // ভুল
}
```

driver হওয়ার কথা ছিল।

Object নিজে থেকে নিশ্চিত করছে না যে:

"প্রতিটি Rickshaw-এর driver, rent, plate থাকতেই হবে।"

❌ 2. একই structure বারবার লিখতে হয়

১০০টা Rickshaw হলে:

Rickshaw 1 → plate, driver, rent...

Rickshaw 2 → plate, driver, rent...

Rickshaw 3 → plate, driver, rent...

...

একই structure বারবার লিখতে হচ্ছে।

❌ 3. Behaviour বাইরে চলে যায়

Rickshaw:

```
const rickshaw = {
  rent: 180
};
```

Daily rent calculate করতে:

```
function dailyTotal(r, days) {  
  return r.rent * days;  
}
```

এখানে **Rickshaw**-এর **data** **Rickshaw**-এর ভিতরে, কিন্তু **Rickshaw**-এর কাজ করার **logic** বাইরে।

আরও function হলে:

```
Rickshaw  
↓  
dailyTotal()  
weeklyTotal()  
calculateTax()  
repairCost()  
...
```

Rickshaw-related knowledge codebase-এর বিভিন্ন জায়গায় ছড়িয়ে যায়।

তাহলে **OOP** কী করতে চায়?

OOP চেষ্টা করে:

```
Data + Behaviour  
↓  
একসাথে রাখা
```

যেমন:

```

class Rickshaw {
  rent: number;

  constructor(rent: number) {
    this.rent = rent;
  }

  dailyTotal(days: number) {
    return this.rent * days;
  }
}

```

Rickshaw

```

├── rent      ← Data
└── dailyTotal() ← Behaviour

```

মনে রাখার সহজ **formula:**

Variables → **data** ছড়িয়ে যায়

Object → **related data** একসাথে করে

OOP → **data + related behaviour properly organize** করে

Loose Variables



"Related data আলাদা আলাদা"



Object



"Related data একসাথে"



কিন্তু নতুন সমস্যা:



1. Structure guaranteed না। Language নিজে থেকে guarantee করছে না যে প্রত্যেক Rickshaw object-এর নির্দিষ্ট property থাকবেই এবং সঠিক নামেই থাকবে।

2. একই structure বারবার লিখতে হয়

3. Behaviour বাইরে ছড়িয়ে যায়



Class + Methods + Encapsulation + অন্যান্য OOP concepts



OOP